CAPSTONE DEFENSE STUDY GUIDE

TechPrime AI

How the system works: logic, flows and the AI parts, explained simply

Client
EasyPC, One Oasis Branch (Rosario, Pasig)

What it is
Inventory, point-of-sale and online-store system with three AI features

Built with
PHP 8.2 · PostgreSQL (Supabase) · vanilla JS/CSS · Python ML services

Based on
The current source code (branch `khenzou`, Oct 2026)

1. The system in one page
   Problem, users, features
2. Technology and architecture
   What runs where, and why
3. How the code is organized
   Folders, page pattern, shared helpers
4. Accounts, login and security
   Register → activate → log in → timeout
5. The five user roles
   Who can do what
6. Online store and the order lifecycle
   Cart → pay → ship → receive → review
7. Cashier point of sale (POS)
   Barcodes, VAT, the safe sale transaction
8. Inventory custodian
   Products, stock-in, alerts, orders
9. Retail officer and admin
   Reports, exports, users, logs, settings
10. AI #1: Sales and demand forecasting
    XGBoost service, reorder point
11. AI #2: Primo chatbot
    TF-IDF + SVM intent model, LLM phrasing
12. AI #3: Build a PC recommender
    Compatibility rules + Random Forest
13. The database
    Main tables and how they connect
14. Notifications, staff chat, deployment
    Supporting features
15. Defense prep: likely questions
    Answers + honest limitations
16. Glossary
    Terms in plain words

How to read this guide

IN SIMPLE WORDS
The idea without jargon.

EXAMPLE
A worked case with numbers.

IF THE PANEL ASKS
A ready answer for the defense.

BE HONEST ABOUT
A limitation to admit, not hide.

Numbers marked “illustrative” are made up to explain a calculation. Every other number (thresholds, accuracy, limits) comes from the code or the model files.

1 The system in one page

TechPrime AI runs one computer store branch end to end: the shelf (inventory), the counter (cashier POS), the website (online store), the office (reports and forecasts) and a few “smart” helpers.

The problem it solves

A computer parts store usually keeps stock in a spreadsheet, sells at the counter with a separate cash register, and has no online shop. Then stock counts drift, nobody knows which items are running out until they are gone, and buying decisions are guesses. TechPrime AI puts all of this in **one database**, so a sale at the counter, an online order and a delivery from a supplier all change the same stock number right away.

Who uses it

ROLE	WHERE THEY WORK	MAIN JOB IN THE SYSTEM
Client (customer)	Online store	Browse, chat with Primo, build a PC, order, pay, review.
Cashier	Store counter	Scan barcodes and sell to walk-in customers, print receipts.
Inventory Custodian	Stock room	Maintain products, receive deliveries (stock-in), process online orders, watch stock alerts.
Retail Officer	Office	Sales reports, sales history, exports, forecasts.
Admin	Office	User accounts, activity (audit) logs, security settings, overview of everything.

The three AI features

1 · Forecasting

Predicts product demand and sales revenue 1 to 3 months ahead using an **XGBoost** model (a separate Python service). Also suggests when to reorder and how much.

2 · Primo chatbot

A shop assistant that understands questions in English and Taglish. An **SVM** model guesses what the customer wants (price? stock? order status?), then real data comes from the database.

3 · Build a PC

Checks that parts fit together (CPU socket, RAM type, case size, power) using rules, then a **Random Forest** ranks the compatible parts and marks the top 3 as “Recommended”.

Plus: barcode POS

Not AI, but a core feature: USB scanner support, check-digit validation, VAT-inclusive receipts and sale transactions that can never oversell.

IN SIMPLE WORDS

One database is the “single source of truth”. Five kinds of users each get their own screens. Three AI helpers sit on top: one predicts the future (forecast), one talks to customers (Primo), one helps customers choose PC parts (Build a PC).

2 Technology and architecture

A classic web application: the browser asks, PHP on the server thinks, PostgreSQL remembers. The AI models run as small helper services the PHP code calls.

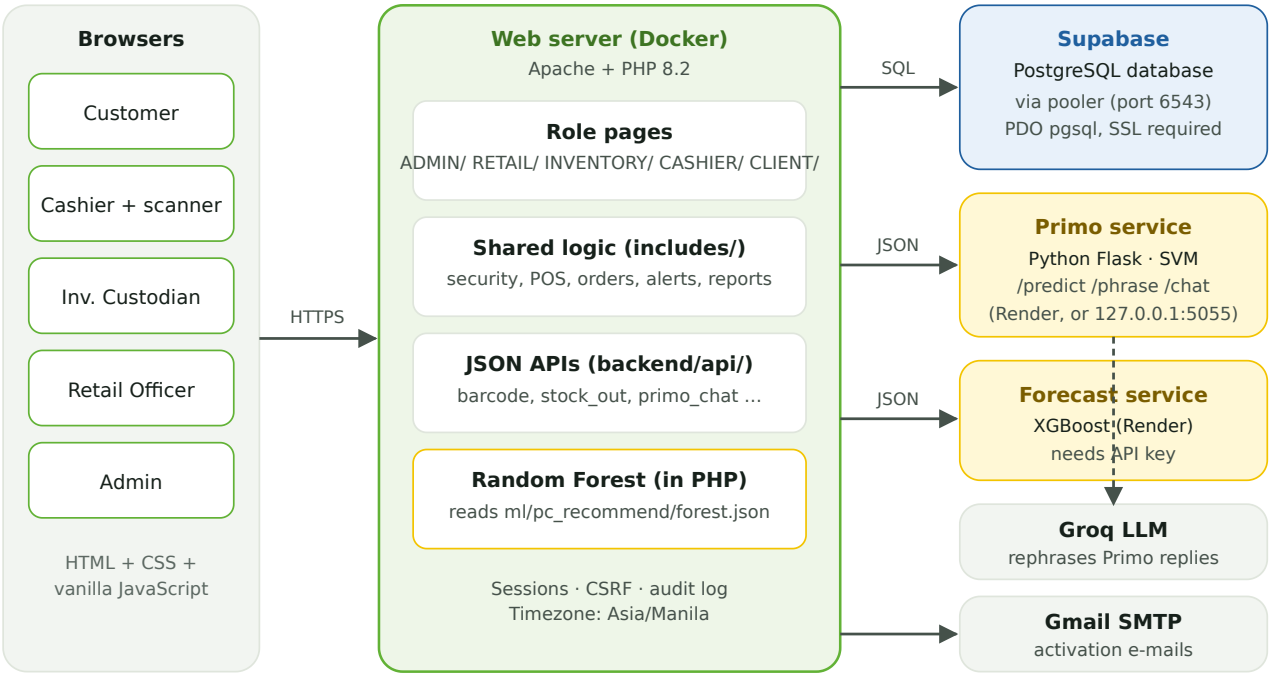


Figure 2.1 · The browser never talks to the database or the AI services directly. Everything goes through PHP.

Each piece and why it was chosen

PIECE	WHAT IT DOES	WHY THIS CHOICE
PHP 8.2 + Apache	Builds every page on the server and handles form posts and JSON requests.	Runs on XAMPP for local work and in a `php:8.2-apache` Docker image for hosting. No build step, easy to deploy.
PostgreSQL on Supabase	Stores users, products, orders, sales, logs and more.	Real transactions and row locks (needed so two cashiers can’t sell the last item twice), hosted and backed up for us.
Vanilla JS + CSS	Live search, the POS cart, the chat widget, charts (Chart.js).	No framework to learn or bundle; pages still work as normal HTML.
Python (scikit-learn, Flask)	Trains the SVM (Primo) and the Random Forest (Build a PC); serves Primo.	Python has the best ML libraries. The forest is exported to JSON so PHP can use it without Python running.
XGBoost forecast service	Demand and revenue forecasts, inventory actions.	Gradient-boosted trees work well on tabular sales data. Hosted separately on Render; PHP reaches it through a proxy that hides the API key.

phpdotenv	Reads secrets (DB password, API keys, SMTP) from a `.env` file or the server environment.	Secrets never sit inside the code or the Git repository.
PHPMailer + Gmail	Sends account-activation e-mails.	Reliable SMTP library; the credentials come from the environment.

IF THE PANEL ASKS: “YOUR PLANNING DOCUMENTS MENTION NEXT.JS AND FASTAPI. WHY PHP?”

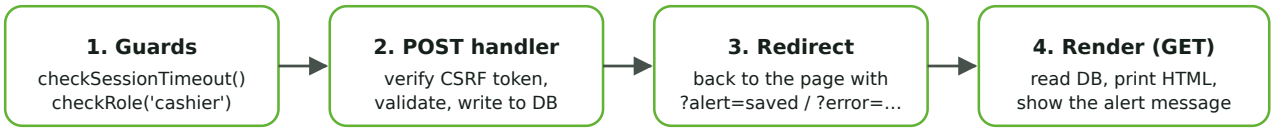
The final build uses plain PHP pages, which fit XAMPP and simple hosting with no build step. The database moved to PostgreSQL on Supabase because the POS needs real transactions and row locking. The machine-learning parts stayed in Python, where the libraries are.

3 How the code is organized

The folder you are in tells you the role. Shared logic lives in `includes/`, so every role uses the same rules.

FOLDER	CONTENTS
login.php, register.php, activitate.php, logout.php	Entry points: sign in, sign up, e-mail activation, sign out.
CLIENT/	The online store: home, shop, product page, cart, checkout, payment, orders, wishlist, notifications, Build a PC, saved builds, Primo chat.
CASHIER/	Dashboard, POS checkout, receipt printing, stock alerts (read only), profile.
INVENTORY/	Custodian pages: dashboard, orders, stocks, stock-in, alerts, messages, forecast, reviews, activity, profile.
RETAIL/	Retail Officer: dashboard, sales history, forecast, stock alerts, messages, exports, profile.
ADMIN/	Admin: dashboard, forecast, stock alerts, messages, manage users, activity logs, settings, profile.
includes/	Shared PHP: `security.php` (login guards, CSRF, logging), `pos_helpers.php` (barcodes, sales, stock-in), `client_helpers.php` (cart, orders), `inventory_alerts.php`, `retail_reports.php`, `pc_compatibility.php`, `pc_recommend.php`, staff layout and chat, plus shared JS (barcode scanner, alerts, session timer).
backend/api/	Small JSON endpoints used by JavaScript: barcode lookup, stock out (sale), stock in, stock alerts, Primo chat, session status.
backend/config/database.php	The one function that opens the database connection: `getDbConnection()`.
ml/primo, ml/pc_recommend	Python training scripts, data and saved models for the two in-house AI models.
scripts/	One-off command-line tools: catalog import, index setup, dataset builders.
database/	SQL migration files (the Postgres ones for POS and chat match the live schema).

The pattern every page follows



Why redirect after a POST? Refreshing the page then repeats a harmless GET, not the form submission, so nothing is saved twice (the “Post/Redirect/Get” pattern).

Figure 3.1 · Request flow inside a typical page.

JSON endpoints (used by JavaScript, like the POS) follow a matching rule: a business problem such as “not enough stock” comes back as HTTP 200 with `{ok:false, error, message}`, so the screen can show the message. Only security problems (not logged in, wrong role, bad CSRF token, wrong method) use 4xx error codes.

4 Accounts, login and security

Customers register themselves and activate by e-mail. Staff accounts are created by the Admin. Everyone signs in on the same login page and is sent to their own area.

Customer registration and activation

1. **Fill in the form** (`register.php`): name, surname, age, phone, full Philippine address (province → city → barangay pickers), e-mail and password. The e-mail and phone are checked live for duplicates.
2. **Password rules** come from the Admin's Settings page (default: at least 8 characters with uppercase, lowercase, a number and a special character).
3. **Saved safely**: the password is hashed with **bcrypt** (`password_hash`). The real password is never stored. The account starts as `is_verified = 0`.
4. **Activation e-mail**: a random 64-character token plus an expiry time (24 hours) is saved and e-mailed as a link. Clicking it (`activitate.php`) sets `is_verified = 1` and deletes the token so the link works only once.
5. **Expired link?** Logging in with the right password opens a "Resend activation link" screen (with a wait time between resends).

Login decision flow

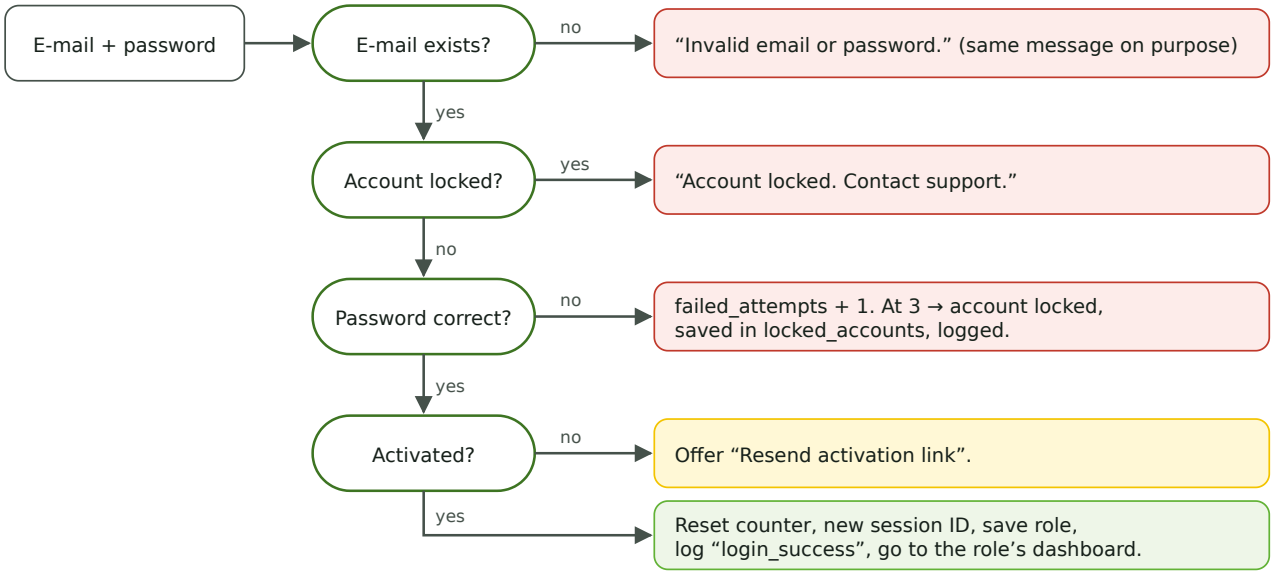


Figure 4.1 · What login.php checks, in order. The password is checked before "activated?" so wrong guesses always count toward the lockout.

Where each role lands after login

USERS.ROLE	REDIRECTED TO
admin	`ADMIN/admin_dashboard.php`
retail_officer	`RETAIL/retail_dashboard.php`

inventory_custodian	`INVENTORY/inventory_dashboard.php`
cashier	`CASHIER/cashier_dashboard.php`
client (or anything else)	The store home page, or the page they came from (for example checkout), if that page is a safe local address.

The security measures, one by one

THREAT	PROTECTION IN THE CODE	IN SIMPLE WORDS
Stolen password database	bcrypt hashes (`password_hash` / `password_verify`)	Even we cannot read the passwords. A hash can be checked but not reversed.
Password guessing	3 wrong tries → `is_locked = 1`; Admin unblocks	A guesser gets three tries, then the door locks.
Guessing which e-mails exist	The same “Invalid email or password” message for both cases	The attacker can’t tell which half was wrong.
Opening another role’s page	`checkRole()` at the top of every page; APIs check `$_SESSION['role']` and answer 403	A cashier typing the admin page address is sent back to login.
Unattended computer	15-minute idle timeout; a warning appears 60 seconds before	Walk away from the counter and the session closes itself.
Session fixation	`session_regenerate_id(true)` on login	A new session ID is issued on login, so an ID planted before login is useless.
Forged form posts (CSRF)	Random token in each form, checked with `hash_equals`	Another website cannot submit our forms, because it doesn’t know the secret token.
SQL injection	PDO prepared statements everywhere	User input is always sent as data, never as part of the SQL command.
Cross-site scripting (XSS)	`h()` = `htmlspecialchars` on every printed value	If someone types `<script>` as their name, it shows as text and never runs.
Untraceable actions	`logActivity()` writes user, action, details and IP to `logs`	Every sale, stock change, login and setting change leaves a trail the Admin can read.

Leaked secrets	`.env` file / environment variables, never in Git	Passwords and API keys are kept out of the code.
Direct database access via Supabase API	Row Level Security switched on with no policies	The public Supabase REST API can read nothing; only our server's connection can.

EXAMPLE · THE IDLE TIMEOUT

A cashier last clicked at 2:00:00 PM. At 2:14:00 a dialog counts down from 60 seconds. If nobody clicks "Stay signed in", at 2:15:00 the session ends and the next click goes to `login.php?expired=1`. The live stock-alert check that runs every 30 seconds deliberately does **not** count as activity, otherwise an open screen would never time out.

5 The five user roles

Each staff role has its own sidebar (on phones, a bottom tab bar). This table is the menu of each role, taken from `staff_nav_for_role()`.

ROLE	MENU ITEMS	CAN CHANGE DATA?
Admin	Dashboard · Forecast (Product Demand, Sales) · Stock Alerts · Messages · Manage Users · Activity Logs · My Profile · Settings	Users (create staff, block, unblock, delete), security settings
Retail Officer	Dashboard · Sales History · Forecast · Stock Alerts · Messages · My Profile	Reads and exports reports only
Inventory Custodian	Dashboard · Orders · Stocks (Inventory, Stock In, Alerts) · Messages · Forecast · Reviews · Activity · My Profile	Products, barcodes, stock-in, order status, cancellations, review replies
Cashier	Dashboard · POS Checkout · Stock Alerts · My Profile	Creates sales (stock-out) only
Client	Home · Shop · Build a PC · Saved Builds · Cart · Wishlist · Orders · Notifications · Primo chat	Own cart, orders, reviews, saved builds

IN SIMPLE WORDS · "SEPARATION OF DUTIES"

The person who sells (cashier) is not the person who receives stock (custodian), and neither can edit user accounts (admin). Stock-in was moved from the cashier to the custodian on purpose: the person handling cash should not also be able to add stock, which would make it easy to hide missing items.

Who sees what (feature matrix)

FEATURE	ADMIN	RETAIL	CUSTODIAN	CASHIER	CLIENT
Forecast pages	✓	✓	✓	–	–
Stock alerts	✓	✓	✓	✓ (read only)	–
Bell notifications for stock	✓	✓	✓	–	own orders
Staff chat (Messages)	✓	✓	✓	–	–
POS sales	–	–	–	✓	–
Stock-in (receiving)	–	–	✓	–	–
Process online orders	–	–	✓	–	places them
Sales reports and exports	dashboard	✓	–	own day	–

User accounts and logs	✓	–	own activity	–	–

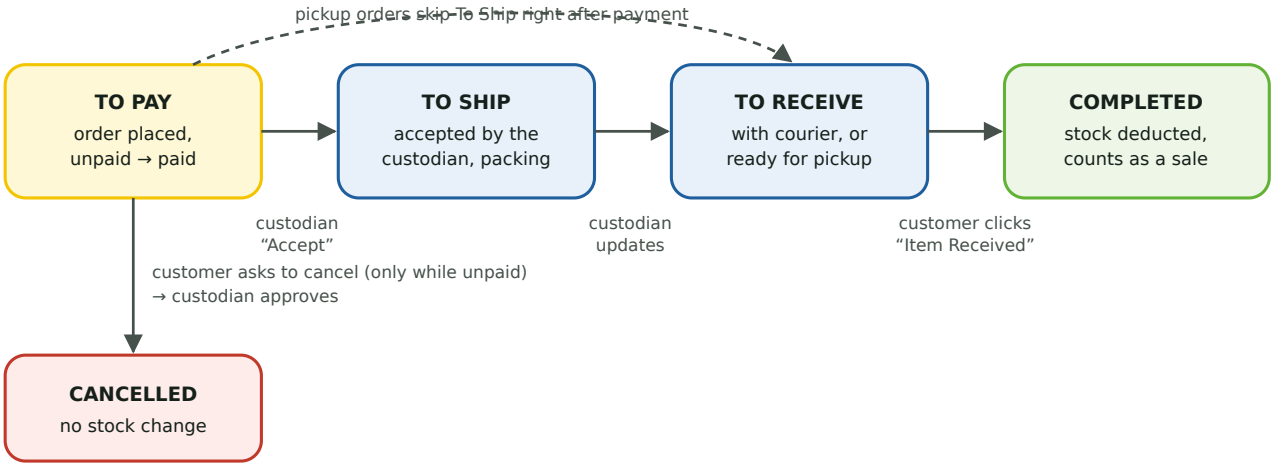
6 Online store and the order lifecycle

An online order moves through fixed stages, like a package tracker. Each stage has a clear owner, and stock is deducted only once, at the very end.

Shopping before the order

- **Browsing**: Home, Shop (with categories, brands, price filters and live search), product detail pages, recently viewed items, wishlist.
- **Cart**: guests have a cart in their session. On login, `ep_sync_cart_on_login()` merges it into the saved database cart (keeping the higher quantity if the item is in both), so a cart survives logout and login.
- **Checkout**: choose a saved address, a new address, or **Pick up at the store**, and give a phone number. Delivery orders are handed to **Lalamove**.
- **Double-submit protection**: a one-time checkout token means clicking “Place order” twice creates only one order.

The order stages



Status only moves forward. Completed and Cancelled are final.

Figure 6.1 · Order stages (database values: to_pay → to_ship → to_receive → to_review, or cancelled).

STEP	WHO	FUNCTION	WHAT HAPPENS IN THE DATABASE
Place order	Customer	`ep_place_unpaid_order()`	Locks each product row and checks stock is enough; creates the `orders` row (status `to_pay`, unpaid) and `order_items`; removes the items from the cart; notifies the customer and the custodian. **Stock is not deducted yet.**

Pay	Customer	`ep_mark_order_paid()`	Sets `payment_status = 'paid'`, a unique payment reference and `paid_at`. Pickup orders jump to `to_receive`. The custodian is told “ready to Accept”.
Accept	Custodian	`ep_accept_order()`	Only if paid and no cancellation pending → `to_ship`; creates a `shipments` row (carrier Lalamove for delivery).
Ship	Custodian	status dropdown	→ `to_receive` (“With Courier”). Moving backward is refused.
Item Received	Customer (or custodian)	`ep_complete_order()`	Deducts stock for every item (refused if stock would go below 0), sets `stock_deducted = TRUE` and status `to_review` (Completed), marks the shipment delivered, notifies the customer, custodian and retail officer.
Review	Customer	`ep_review_submit()`	One review per completed order: 1–5 stars, comment, tags, optional photos, can be anonymous. The custodian can reply.
Cancel	Customer → Custodian	`ep_request_order_cancel()`, `ep_accept_order_cancel()`	Allowed only while the order is `to_pay` and **unpaid**. The customer picks a reason; the custodian approves. No stock change, because none was deducted.

IN SIMPLE WORDS · WHY DEDUCT STOCK ONLY AT “COMPLETED”?

An order can still be cancelled before it is paid, so taking stock away at order time would make items look sold out when they are not. The stock is checked (and locked) when the order is placed, and actually removed when the customer has the item. The `stock_deducted` flag makes sure it happens only once, even if the button is clicked twice.

EXAMPLE · ONE ORDER FROM START TO END (ILLUSTRATIVE)

1. Ana orders 1 × 16 GB RAM kit (stock 9). The order becomes **#ORD-120, To Pay**. Stock is still 9.
2. Ana pays. The custodian’s bell shows “Paid order #ORD-120 ready to Accept”.
3. The custodian clicks Accept → **To Ship**, packs it, and sets it to **To Receive** when Lalamove picks it up.
4. Ana clicks **Item Received** → stock 9 → 8, order **Completed**. The sale now appears in the Retail Officer’s reports.
5. No new stock alert: 9 and 8 are both in the “low” level (6–15), and alerts fire only when the level changes. Had the stock gone from 16 to 15, every stock-alert user would get “low in stock (15 left)”.

BE HONEST ABOUT · ONLINE PAYMENT

The **Pay** page is a placeholder for the school project (the page itself says so). Clicking “Pay Now” marks the order paid with a generated reference. No real money moves and no payment gateway is called. A real gateway such as PayMongo would replace only this one step; the rest of the order flow stays the same.

7 Cashier point of sale (POS)

The counter screen for walk-in customers: scan, review the cart, take payment, print the receipt. The important part is what happens behind “Complete sale”.

The flow at the counter

1. **Scan**. A USB barcode scanner works like a very fast keyboard: it “types” the digits and presses Enter. `barcode_scanner.js` spots this (keys arriving a few milliseconds apart), so scanning works without clicking into a field first.
2. **Validate the check digit** in the browser and again on the server (see the example below). A misread code is rejected right away.
3. **Look up the product** (`backend/api/barcode_lookup.php`): name, price and stock come back and the item is added to the cart.
4. **Choose payment**: Cash (enter the amount tendered, change is computed), GCash, Maya or Card (a reference number is required).
5. **Complete sale** (`backend/api/stock_out.php` → `pos_complete_sale()`): one database transaction does everything, or nothing.
6. **Receipt** opens with the invoice number (format `INV-YYYYMMDD-000123`), VAT breakdown and change, and prints in the same tab.

Barcode check digit, worked example

Every UPC/EAN barcode ends with a **check digit** computed from the other digits. If one digit is misread, the check digit no longer matches. The system accepts UPC-A (12 digits), UPC-E (8), EAN-13 (13) and EAN-8 (8).

EXAMPLE · IS 4800123456786 A VALID EAN-13?

DIGIT	4	8	0	0	1	2	3	4	5	6	7	8	CHECK
WEIGHT	1	3	1	3	1	3	1	3	1	3	1	3	–
PRODUCT	4	24	0	0	1	6	3	12	5	18	7	24	**6**

Sum = 104. Check digit = (10 − (104 mod 10)) mod 10 = (10 − 4) mod 10 = **6**. The last digit is 6, so the code is valid. If the scanner misread one digit, for example 480012345679**9**6, the sum becomes 107, the expected check digit becomes 3 instead of 6, and the code is rejected with “Invalid barcode check digit”.

The code stores every barcode in one normalized form (12-digit UPC-A gets a leading 0 to become 13 digits; UPC-E is expanded first), so the same product is found whichever format is scanned. Products without a barcode can get an in-store one: `21` + the 10-digit product ID + a check digit (product #57 → 2100000000579), printable as a label.

What “Complete sale” does inside the database

ONE TRANSACTION: all steps succeed together, or none are saved

1 · Already done?
Look up client_ref (a UUID). If found, return that sale again.

2 · Lock products
SELECT … FOR UPDATE, always in product-ID order.

3 · Re-check stock
qty ≤ stock for every line, or stop: “Not enough stock”.

4 · Price from the DB
Never trust the browser’s price. Totals in integer centavos.

5 · Payment check
Cash ≥ total (change computed); e-wallet/card needs a reference.

6 · Write the sale
pos_sales (with VAT split) + pos_sale_items.

7 · Deduct stock
UPDATE … SET stock = stock − qty WHERE stock ≥ qty (extra guard)

8 · Movement + log
stock_movements (before → after) + logs entry “pos_sale”.

9 · Commit
Then (outside the transaction) send low-stock alerts.

Any error at any step → ROLLBACK → “The sale could not be completed. No changes were saved.”

Figure 7.1 · `pos_complete_sale()` in includes/pos_helpers.php.

WHY LOCK THE ROWS? (OVERSELLING)

Two cashiers sell the **last** mouse at the same second. Without a lock, both read “stock 1” and both sell. With `FOR UPDATE`, the second cashier waits until the first finishes, then reads “stock 0” and gets “Not enough stock”. The database rule `CHECK (stock >= 0)` is a last safety net.

WHY LOCK IN ID ORDER? (DEADLOCK)

If sale A locks mouse then keyboard while sale B locks keyboard then mouse, each waits for the other forever. Locking always in the same order (lowest ID first) makes that impossible.

WHAT IS CLIENT_REF? (IDEMPOTENCY)

The POS screen creates a random ID (UUID) for each sale before sending it. If the Wi-Fi drops and the cashier clicks again, the server sees the same ID and returns the first sale instead of charging twice. The column is `UNIQUE`, so even two requests at the same instant cannot both succeed.

WHY INTEGER CENTAVOS?

Computers store decimals like 0.1 inaccurately. ₱1,234.50 is handled as 123450 centavos, so totals and change are always exact.

EXAMPLE · VAT ON A RECEIPT (ILLUSTRATIVE)

Prices already include 12% VAT. The cart total is **₱3,450.00** (345,000 centavos).

- VATable sales = 345,000 ÷ 1.12 = 308,035.7 → rounded to **₱3,080.36**
- VAT = total − VATable = 345,000 − 308,036 = **₱369.64**
- Customer pays ₱4,000.00 cash → change = **₱550.00**

VAT is total minus VATable (not a separate rounding), so the two always add up to the total exactly.

Other cashier screens

- **Dashboard**: today’s sales count and total for this cashier, recent sales, a live stock-alert panel (refreshes every 30 seconds).
- **Stock Alerts**: out / critical / low products, read only, so the cashier can tell customers what is running out.
- Limits: at most 999 per line and 200 different products per sale; references are letters, digits, spaces and `- / #`, up to 64 characters.

8 Inventory custodian

The custodian owns the product list and the stock numbers, receives supplier deliveries and moves online orders forward.

Products (Stocks page)

- Add, edit, delete (single or bulk) products: name, category, price, stock, description, image, **SKU** (EasyPC's internal item code) and **barcode**.
- The catalog was imported from EasyPC's item list (`Oasis-Itemlist.csv`, 261 products) with images per category.
- Print a barcode label, or generate an in-store barcode for products that have none.

Stock-in (receiving a delivery)

1. Scan the product's barcode (same scanner logic as the POS).
2. Enter the quantity (1 to 10,000), choose the **supplier** (required) and an optional reference number (for example the delivery receipt number).
3. `pos_stock_in()` locks the product row, adds the quantity, writes a `stock_movements` row of type `stock_in` (with stock before and after, supplier, reference) and an audit log line like `stock 3 → 23 (+20), supplier: Kingston`.
4. If the product was out or low and is now fine, the alert roles get "stock increased by 20 (now 23)".

BE HONEST ABOUT · SUPPLIERS

There is no separate suppliers table. A supplier is the product's **brand**: the first word of the product name, matched against the list of shop brands (for example "Kingston FURY 16GB…" → Kingston). It keeps receiving simple, but a distributor that sells several brands can't be recorded as itself.

Stock alert levels

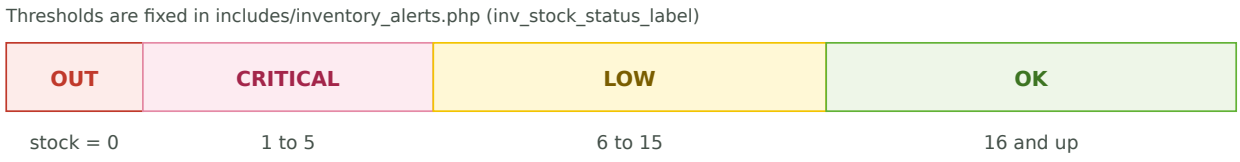


Figure 8.1 · How a stock number becomes a status. Status is always shown with an icon and text, not colour alone.

Alerts fire on a change of level, not on every sale. `inv_notify_stock_change(old, new)` runs after every stock change (POS sale, online order completion, stock-in) and sends a bell notification to every Admin, Retail Officer and Custodian only when the stock *crosses* into Low, Critical or Out, or recovers. The same alert is not repeated to the same person within 24 hours.

EXAMPLE · CROSSING LEVELS

Stock 16 → 15: “low in stock (15 left)”. 15 → 14: nothing new. 6 → 5 after a sale: Low to Critical, both “running low”, so nothing new is sent (the Stock Alerts page still shows it as Critical). 1 → 0: “out of stock”. Then a stock-in of 20 (0 → 20): “stock increased by 20 (now 20)”.

Orders, reviews and activity

- **Orders**: tabs by stage, Accept (single or bulk), status dropdown that only moves forward, cancellation requests to approve. Completed and cancelled orders are locked.
- **Reviews**: all customer reviews with stars, photos and tags; the custodian can reply.
- **Activity**: the custodian’s own audit trail (stock-ins, product edits, order updates, stock-outs), showing stock changes as “before → after”.
- **Forecast**: the same forecast pages as Admin and Retail, including the reorder suggestions (chapter 10).

9 Retail officer and admin

The office roles mostly read data: the Retail Officer turns sales into reports, the Admin manages people and settings and watches the audit trail.

What counts as a “sale” in reports

IN SIMPLE WORDS

Sales = **completed online orders** (status Completed) **+ POS sales** at the counter. Orders that are unpaid, in transit or cancelled do not count. Reports are read live from the database, so a sale shows up the moment it is completed.

Retail Officer

- **Dashboard**: total sales, transactions, units sold and average transaction for a chosen period (presets like Today, This week, This month, or a custom range), sales by category, a daily trend chart, a 12-month breakdown and the forecast cards.
- **Comparisons**: each figure is compared with the previous period (month-over-month) and the same period a year ago (year-over-year), shown as a green ▲ or red ▼ badge.
- **Sales History**: every sale line, searchable and filterable by category and customer.
- **Exports**: CSV, Excel (.xlsx) or PDF, with a “prepared by” line. Each export is written to the audit log.

EXAMPLE · THE % CHANGE BADGE (ILLUSTRATIVE)

This month ₱120,000, last month ₱100,000 → (120,000 − 100,000) ÷ 100,000 × 100 = ▲ **20.0%**. If last month was ₱0, the badge says “new” instead of dividing by zero.

Admin

- **Manage Users**: counts per role, locked and unverified accounts; create staff accounts (retail officer, inventory custodian, cashier), which are verified immediately; **block / unblock** (unblocking clears the lockout) and delete accounts.
- **Activity Logs**: the full audit trail from the `logs` table, such as logins, lockouts, sales, stock-ins, order actions, exports, settings changes, with the user, time and IP address.
- **Settings**: password policy (minimum length 6–32, upper, lower, number, special). New passwords must follow it.
- **Stock Alerts, Forecast, Messages**: same shared pages as the other office roles.

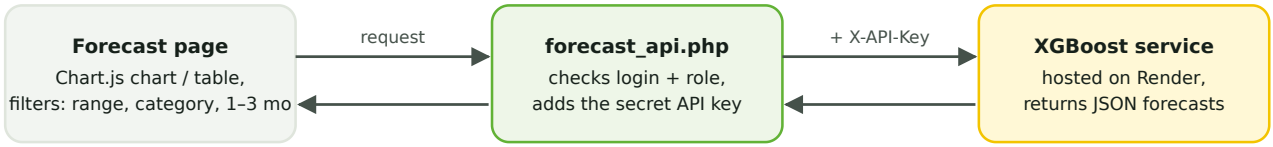
BE HONEST ABOUT · “MAX FAILED ATTEMPTS” SETTING

The Settings page saves a “max failed attempts” value (1–10), but `login.php` currently locks after a fixed **3** attempts. If asked: the lockout works at 3; connecting the setting to the login check is a small, known follow-up.

10 AI #1 · Sales and demand forecasting

Looks at past monthly sales and predicts the next 1 to 3 months: how many units of each product will sell (demand) and how much money the store will make (revenue).

How a forecast reaches the screen



The API key never reaches the browser. Inputs are validated (horizon 1–3, safe category text, real dates).

Figure 10.1 · The PHP proxy pattern. Admin, Retail Officer and Inventory Custodian can open it; other roles get 403.

What XGBoost is, in plain words

IN SIMPLE WORDS

XGBoost builds many small decision trees **one after another**. The first tree makes a rough guess of next month's sales; the second tree learns the mistakes of the first; the third fixes what is still wrong, and so on. Added together, hundreds of small corrections give a strong prediction. It learns from patterns such as recent months' sales, trend and season, per product and category.



Illustrative numbers. Each tree corrects the error left by the trees before it ("boosting").

Figure 10.2 · Boosting: a forecast is the sum of many small corrections.

What the forecast pages show

VIEW	COLUMNS / CONTENT
Product Demand	Units sold in the chosen date range, forecast units, a likely range (low–high), forecast revenue, trend (month-over-month growth), confidence and model reliability. Filter by category.
Inventory actions	Current stock, days of inventory left, safety stock, reorder point, stockout risk and a recommended order quantity, with an action such as "order now" or "reorder". Shown once the service has a stock snapshot; otherwise a note explains why the columns are hidden.
Sales	Store-wide revenue: actual months next to forecast months, with a confidence band (low–high).

EXAMPLE · REORDER POINT AND SAFETY STOCK (TEXTBOOK DEFINITIONS, ILLUSTRATIVE NUMBERS)

A product is forecast to sell **60 units next month ≈ 2 per day**. The supplier takes **7 days** to deliver. Keep an extra **6 units** of *safety stock* for surprises.

- **Reorder point** = daily demand × lead time + safety stock = 2 × 7 + 6 = **20**. When stock falls to 20, order more.
- **Days of inventory** = stock ÷ daily demand. With 30 in stock: 30 ÷ 2 = **15 days** left.

These are the standard formulas the columns are named after. The exact calculation runs inside the forecast service, which lives in its own repository, not in this codebase.

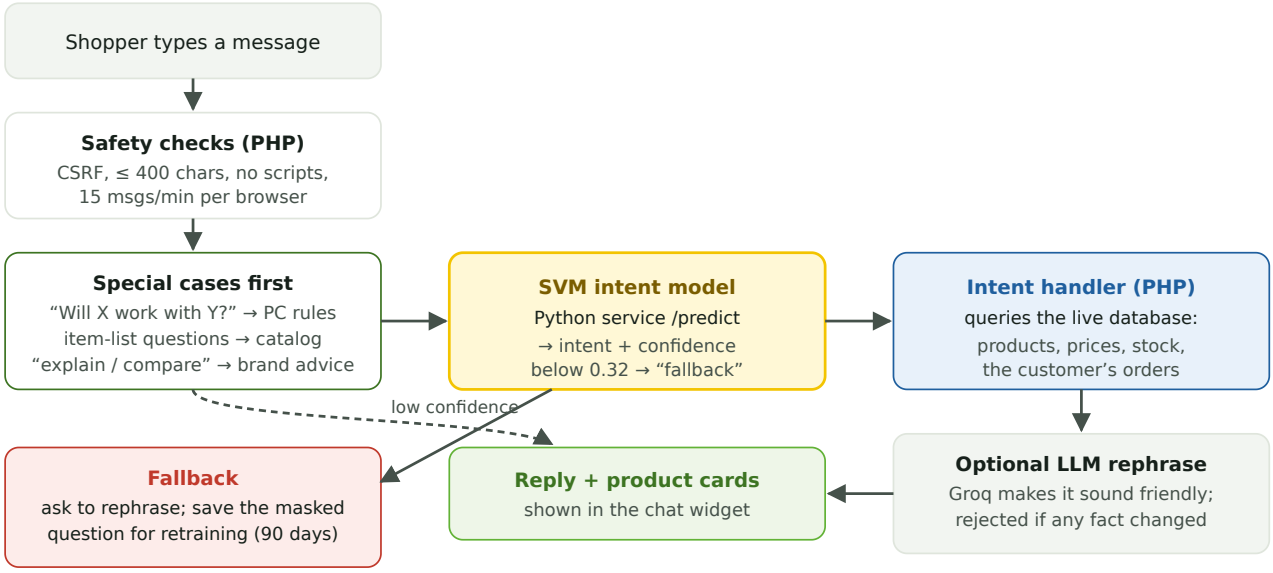
IF THE PANEL ASKS: “WHY IS THE FIRST FORECAST SLOW SOMETIMES?”

The forecast service runs on Render’s free tier, which sleeps when idle and needs up to about 50 seconds to wake. The proxy waits up to 75 seconds, and the page tells the user to retry in a minute if it’s still waking up.

11 AI #2 · Primo chatbot

Primo answers shopper questions like “magkano yung ryzen 7?” or “is the RTX 4060 in stock?”. A machine-learning model decides *what kind* of question it is; the *answer* always comes from the live database.

The pipeline



Off-topic messages (homework, dating, weather…) get a fixed “I’m only here for EasyPC” reply and are never sent to the LLM.

Figure 11.1 · How backend/api/primo_chat.php answers one message.

The intent model: TF-IDF + SVM

An **intent** is the goal behind a message. Primo knows 16: greeting, goodbye, gratitude, help, product_search, product_category, product_price, stock_inquiry, product_recommendation, brand_advice, compatibility_question, saved_build, order_status, return_refund, store_information, and fallback (not understood).

1. **Clean the text**: lowercase, normalize spelling.
2. **TF-IDF** turns the sentence into numbers. It counts words and word pairs (“in stock”), plus 3–5-letter chunks (“mag”, “agka”, “kano”), and gives more weight to words that are rare across all examples. The letter chunks are why typos and Taglish still work.
3. **SVM** (Support Vector Machine, RBF kernel) has learned boundaries that separate the 16 intents in that number space, and outputs a probability for each.
4. **Threshold**: if the top probability is below **0.32**, the answer is “fallback” (ask to rephrase) instead of a risky guess.

EXAMPLE · “MAGKANO YUNG RYZEN 7?” (ILLUSTRATIVE PROBABILITIES)

INTENT	PROBABILITY
product_price	**0.71**
product_search	0.12
stock_inquiry	0.08
others	0.09

0.71 ≥ 0.32 → intent **product_price**. The PHP price handler searches products for “ryzen 7” and replies with real rows, for example “AMD Ryzen 7 … — ₱(price from DB) — (n) on hand”. A follow-up like “how about stock?” reuses the last product topic saved in the session.

Measured accuracy (from ml/primo/BASELINE.md)

MEASURE	RESULT	WHAT IT MEANS
Hard test set (160 tricky sentences: typos, Taglish, new phrasing; never used for training)	**83.8%** (134/160)	The honest “real-world” score. Up from 59.4% after adding Taglish and slang examples.
5-fold × 5 cross-validation	82.7% ± 1.7	Train on 4/5 of the data, test on the other 1/5, repeated 25 times.
Quick checks	47/47	Common sentences that must always work.
Training data	922 examples	102 in Taglish/Filipino; every intent has at least 40.

Why the LLM can’t make things up

IN SIMPLE WORDS

The LLM (Groq, model `gpt-oss-20b`) is only allowed to **rephrase** a sentence that PHP already built from the database. After it answers, `primo_phrase_keeps_facts()` checks that every peso amount is the same, no new peso amount appeared, the product name is still there and every “N on hand” is unchanged. If anything differs, the original database sentence is used. If Groq is off, Primo still works with the plain sentences.

Privacy and abuse protection

- E-mail addresses and phone numbers are masked before anything goes to the LLM or into the “unanswered questions” log.
- The unanswered log keeps only the masked message text (no user ID, session or IP), at most 30 per visitor, deleted after 90 days. Staff review it and add good examples to the training data: the model learns from real questions.

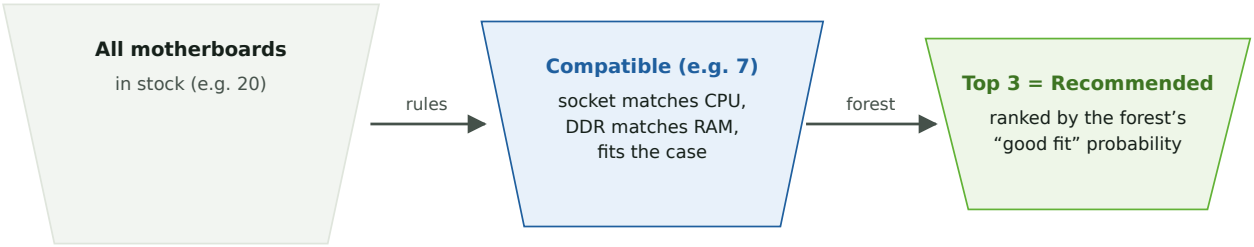
- Rate limits: 15 messages per minute per browser, 60 per IP address.

BE HONEST ABOUT

“help” and “fallback” are the weakest intents (about 4–6 out of 10 on the hard set), and the test sentences were written by the same team as the training data, so they aren’t fully independent. Real shopper messages from the unanswered log are the planned next source of data.

12 AI #3 · Build a PC recommender

The customer picks a purpose (School, Office, Gaming, Content Creation, or Not sure yet), an optional budget, and then a part for each slot. Two layers help: **rules** remove parts that won't fit, then a **Random Forest** ranks the parts that remain.



The forest never sees incompatible parts; rules are the safety layer, the model is the taste layer.

Figure 12.1 · Two-stage recommendation for one slot (illustrative counts).

Layer 1 · Compatibility rules (includes/pc_compatibility.php)

Each product's name and description are read for tags such as socket (AM4, AM5, LGA1700), memory type (DDR4/DDR5), form factor (ITX, mATX, ATX, E-ATX), storage type (NVMe/SATA) and wattage. Then:

RULE	MESSAGE THE CUSTOMER SEES
CPU socket = motherboard socket	"CPU socket mismatch"
RAM type = motherboard type (and what the CPU supports)	"RAM type mismatch" / "this processor needs DDR5"
Cooler socket fits the CPU/motherboard	"cooler socket mismatch"
Case size ≥ motherboard size (ITX < mATX < ATX < E-ATX)	"case form factor too small"
PSU watts ≥ estimated draw (CPU 65 W + board 40 W + GPU 180 W + RAM 10 W + storage 15 W)	"PSU wattage too low for estimated power draw"

An extra check refuses any part that would leave another **required** part with **no compatible option in stock** (a "dead end").

Layer 2 · The Random Forest

IN SIMPLE WORDS

A decision tree is a flowchart of yes/no questions ("Is this part much pricier than the rest of the build?", "Does the purpose need a graphics card?"). A **Random Forest** is **60 such trees**, each trained on a slightly different random sample. Each tree votes how good a fit the part is; the forest averages the votes. Many imperfect voters together are more reliable than one.

EXAMPLE · SCORING ONE GRAPHICS CARD FOR A GAMING BUILD (ILLUSTRATIVE)

Tree 1 says 0.9, tree 2 says 0.7, tree 3 says 0.8 … the average of all 60 is **0.78**. That is above the threshold 0.5, so it is a “good fit”. Among all compatible GPUs, the three highest averages get the **Recommended** badge with a reason (“Fits the parts you already picked”). A part that fits the rules but scores low gets a note such as “Pricier than the rest of this build needs.”

What the trees look at (23 features): the purpose, which slot, how the part’s price tier compares with the rest of the build, overall build level, GPU level, performance tier, balance, cooling margin, dual-channel memory, PSU headroom and rating, ATX 3.0, case bundle, form-factor slack, brand match, built-in graphics, bundled cooler, VRAM and budget slack. The most important ones are **budget slack (19%)**, slot (14%), price-tier gap (13%) and performance tier (11%).

Where the training data comes from

- **Rule rows (53,985)**: random partial builds from the real catalog, for every purpose and several sample budgets (₱30k to ₱200k). Each candidate part is labelled good/not good by expert rules.
- **Real rows (739)**: real customer choices from saved builds, online orders and POS sales. One part is hidden; the part the customer actually chose is “good”. Real picks count 5× more, so the model follows customers where they disagree with the rules.

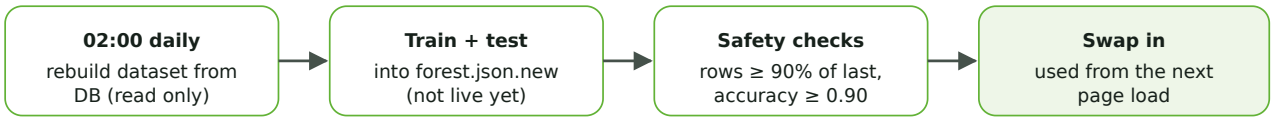
How well it works (from models/forest.json)

TEST	RESULT	MEANING
Accuracy, 4-fold cross-validation grouped by build	**99.0%**	Agrees with the labels on builds it never saw (F1 0.989).
Hit@3 on real customer builds	**65.1%**	Hide a part a real customer chose: in 65% of cases it is in the model’s top 3.
Same test, expert rules alone	57.0%	The model beats every simpler method, which shows the machine learning adds value over plain rules.
Same test, closest price range	51.2%	
Same test, random picks	39.9%	

BE HONEST ABOUT

The 99% is high partly because most rows are labelled by rules, which a forest learns easily. On the 739 real rows alone accuracy is about 47%, because a customer’s single choice is “right” and every other good part counts as “wrong”. That’s why Hit@3 (is the real choice in the top 3?) is the fairer measure. Compatibility also depends on product names and descriptions being written clearly.

Nightly self-improvement (ml/pc_recommend/retrain.py)



If any check fails, the current model stays live and the reason is written to logs/retrain.log.

Figure 12.2 · Automatic retraining that can never put a broken model online.

“Build it for me” uses the same pieces: given a purpose and budget, it fills each empty slot in order (CPU, motherboard, memory, GPU if the purpose needs one, storage, PSU, case, cooler, fans), using a budget share per slot for that purpose and the best-scoring compatible part. Builds can be saved to the account and added to the cart.

13 The database

PostgreSQL on Supabase. These are the tables that matter most and how they connect.

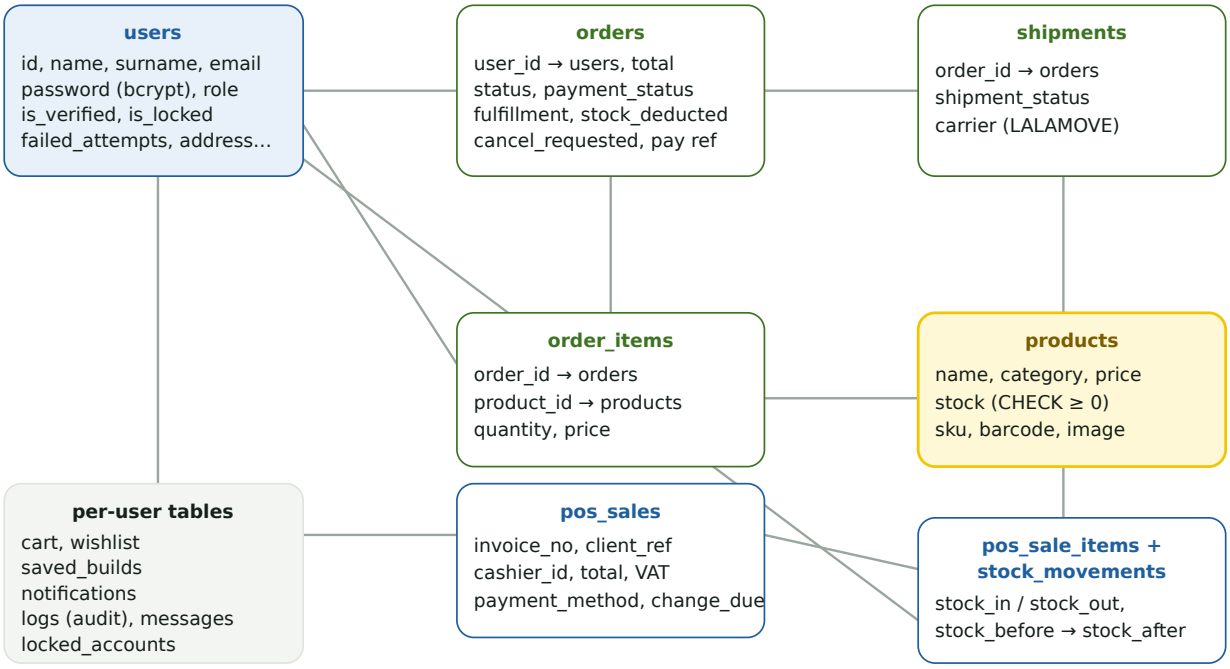


Figure 13.1 · Simplified entity relationships. Arrows in the text (→) are foreign keys.

TABLE	HOLDS
users	Every account and its role. `role` is limited by a CHECK constraint to the valid roles.
products	The catalog and the live stock number. `stock` can never go below zero (database CHECK).
orders, order_items, shipments	Online orders, their lines and delivery status.
pos_sales, pos_sale_items	Counter sales and their lines; names and prices are copied at the time of sale, so old receipts never change.
stock_movements	The stock ledger: every stock-in and POS stock-out with before/after values, supplier, reference, user.
logs	The audit trail: user, action, details, IP, time.
notifications	Bell items for customers (order updates) and staff (stock alerts, new orders).
order_reviews, order_review_media	Customer reviews, photos and staff replies.
saved_builds, cart, wishlist	Customer data kept between visits.

messages (+ reactions, pins, hidden)	Staff chat.
site_settings	Admin-controlled settings (password policy).
primo_unanswered	Masked questions Primo could not answer, for retraining.

SMALL DETAILS WORTH KNOWING

- **Time**: timestamps are saved in UTC and shown in Philippine time (Asia/Manila), so reports are correct regardless of the server's location.
- **SKU vs barcode**: SKU is EasyPC's own item code; the barcode is the manufacturer's UPC/EAN, stored in one normalized form.
- **Connection**: through Supabase's connection pooler (transaction mode), with SSL required. Each piece of work stays inside one transaction.
- **Outage page**: if the database can't be reached, visitors see a friendly "can't connect" page; technical details go only to the server log.

14 Notifications, staff chat, deployment

The features that tie the roles together and keep the system running.

Notifications (the bell)

WHO	GETS NOTIFIED ABOUT
Customer	Order placed, payment successful, order accepted, completed, cancellation updates.
Inventory Custodian	New and paid orders, cancellation requests, items received, plus all stock alerts.
Admin, Retail Officer	Stock alerts (low, out, restocked); the Retail Officer also gets “order completed, recorded as a sale”.

Clicking an item opens the right page already filtered, for example a low-stock alert opens Stock Alerts showing only items that are running low.

Staff chat (Messages)

An internal messenger for Admin, Retail Officer and Inventory Custodian (not cashiers). It supports one-to-one conversations, file and image attachments, reactions, unsend, forward, pin, search and read status. It also appears as a floating widget on every staff page. Every action checks that the user really belongs to that conversation, and attachments are stored in a folder that can’t be opened directly from the web.

Deployment

- **Local**: XAMPP (Apache + PHP) at `http://localhost/TechPrime-AI/TechPrime-AI/login.php`, with Primo’s Python service on `127.0.0.1:5055`.
- **Hosted**: a Docker image based on `php:8.2-apache` with the PostgreSQL driver, timezone set to Asia/Manila and Composer dependencies installed during the build. Secrets come from environment variables.
- **AI services**: Primo (Flask) and the XGBoost forecast service run on Render. The Build-a-PC model is a JSON file read by PHP, so it needs no extra server.

The look (staff design system)

All staff screens share one app shell, modelled on the Cashier redesign: a dark green sidebar on desktop, a bottom tab bar on phones, EasyPC green and yellow, Fira Sans/Fira Code fonts, buttons at least 44 px tall, and tables that turn into cards on small screens. Status always uses an icon and text, not colour alone. The customer store has its own matching design with a light/dark theme switch.

15 Defense prep: likely questions

Short, honest answers. Say what the system does, how you know it works, and what you would improve.

Q: What makes the system “AI”?

> **A:** Three trained models: an XGBoost model forecasts demand and revenue; an SVM classifies the intent of chatbot messages; a Random Forest ranks compatible PC parts. Each was evaluated: Primo 83.8% on a held-out hard test set, the forest 99.0% cross-validated accuracy and 65% Hit@3 on real builds, better than rules alone (57%).

Q: How do you prevent selling an item that is already out of stock?

> **A:** At the POS, the sale locks the product rows (`FOR UPDATE`), re-checks stock inside the transaction, deducts with `WHERE stock >= qty`, and the database itself forbids negative stock. If any check fails, the whole sale rolls back. Online orders check and lock stock when placed and deduct once at completion.

Q: What if the cashier clicks “Complete sale” twice or the network drops?

> **A:** Each sale carries a unique `client_ref` created by the POS screen. A repeated request returns the same sale instead of a new one; the UNIQUE constraint covers even simultaneous requests.

Q: Why is the stock of an online order not deducted when the order is placed?

> **A:** The order can still be cancelled while unpaid. Stock is validated at placement and deducted exactly once, when the customer confirms receipt (or staff completes it), guarded by the `stock_deducted` flag.

Q: How are passwords protected?

> **A:** bcrypt hashes, an admin-configurable complexity policy, lockout after 3 wrong attempts, identical error messages for wrong e-mail or password, a new session ID at login and a 15-minute idle timeout.

Q: How do you stop one role from opening another role’s pages?

> **A:** Every page starts with `checkSessionTimeout()` and `checkRole()`; JSON endpoints check the session role and return 403. Typing another role’s URL redirects to login.

Q: Can the chatbot give a wrong price?

> **A:** Prices and stock always come from the database. The LLM only rephrases, and the reply is thrown away if any price, product name or quantity changed. If the model is unsure (below 0.32 confidence) Primo asks the customer to rephrase instead of guessing.

Q: How does Primo understand Taglish and typos?

> **A:** TF-IDF uses 3–5 character chunks as well as words, so “magkano”, “mgkano” and “magkanoo” share most chunks. The training data includes 102 Taglish/Filipino examples.

Q: How does the forecast help the business?

> **A:** It shows which products will sell in the next 1–3 months and turns that into inventory actions: reorder point, safety stock, days of inventory left and a recommended order quantity, so buying decisions are based on data.

Q: How does the Build-a-PC know two parts are compatible?

> **A:** It reads the socket, memory type, form factor, storage type and wattage from product names and descriptions, then applies fixed rules (socket match, DDR match, case ≥ board size, PSU ≥ estimated draw). Only parts that pass are ranked by the model.

Q: Does the model keep learning?

> **A:** Yes. The Build-a-PC forest retrains every night from new saved builds and sales, and goes live only if it passes the data-size and accuracy checks. Primo's unanswered questions are reviewed and added to its training data.

Q: How is every action traceable?

> **A:** `logActivity()` records the user, action, details (for stock: before → after) and IP for logins, sales, stock-ins, order actions, exports and settings changes. Admin sees all of it in Activity Logs.

Q: What happens if the database or an AI service is down?

> **A:** Database down: a branded "can't connect" page, details only in the server log. Primo down: "temporarily unavailable" message. Groq down: Primo still answers with plain database sentences. Forecast asleep: a "waking up, retry" message. Build a PC without the model file: the rules still filter parts.

Limitations to admit (and the fix you would make)

LIMITATION	NEXT STEP
Online payment is a placeholder: no real money moves.	Connect a gateway (e.g. PayMongo) and mark orders paid from its confirmation.
The "max failed attempts" setting is saved, but login always locks at 3.	Read the setting in `login.php`.
Stock thresholds (5 / 15) are the same for every product.	A per-product reorder level, or use the forecast's reorder point.
Supplier = product brand; no suppliers table.	Add a suppliers table linked to products.
PC compatibility is read from product text, so it depends on well-written names and descriptions.	Store specs (socket, DDR, form factor, watts) as proper product columns.
Two-factor (TOTP) code exists in the old API (`backend/api/auth.php`) but the main login page doesn't use it.	Add the TOTP step to `login.php` for staff accounts.
AI services on free hosting sleep when idle (slow first request).	A paid plan or a scheduled warm-up.
One shared live database; no separate test database.	A staging database for testing.

16 Glossary

Terms you may hear or use during the defense.

Audit trail / log	A record of who did what and when (`logs` table).
bcrypt / hash	A one-way scramble of a password. You can check a password against it, but you can’t reverse it.
Check digit	The last digit of a barcode, computed from the others to catch misreads.
Confidence threshold	The minimum probability before Primo trusts its guess (0.32).
Cross-validation	Testing a model on parts of the data it didn’t train on, several times, and averaging.
CSRF token	A secret value in each form so other websites can’t submit forms as the user.
Deadlock	Two processes each waiting for a lock the other holds. Avoided by locking in a fixed order.
F1 score	One number that balances precision (how many “yes” answers were right) and recall (how many real “yes” cases were found).
Hit@3	How often the correct answer is among the top 3 suggestions.
Idempotent	Doing the same request twice has the same effect as doing it once.
Intent	The purpose behind a chat message (price, stock, order status…).
LLM	Large Language Model, a text-generating AI (used here only to rephrase).
PDO / prepared statement	PHP’s database layer; user input is sent separately from the SQL so it can’t be run as code.
Random Forest	Many decision trees that vote; the average vote is the prediction.
Reorder point	The stock level at which you should order more (daily demand × lead time + safety stock).
Row lock (FOR UPDATE)	Reserves a database row so no one else can change it until the transaction ends.
Safety stock	Extra units kept for unexpected demand or late deliveries.
Session	The server’s memory of a logged-in user, linked to a browser cookie.
SKU	Stock Keeping Unit, the store’s own product code.
SVM	Support Vector Machine, a model that learns boundaries between categories.

TF-IDF	Turns text into numbers; rare, distinctive words weigh more than common ones.
Transaction / rollback	A group of database changes that are saved together, or all undone if one fails.
UPC / EAN	Standard product barcode formats (12 / 13 / 8 digits).
UUID	A random, practically unique ID (used as the POS `client_ref`).
VAT-inclusive	The shelf price already contains the 12% VAT; VAT = total − total ÷ 1.12.
XGBoost	Gradient-boosted trees: each new tree corrects the errors of the previous ones.

Good luck at the defense. If a question goes deeper than this guide, the file names in each chapter point to the exact code to open.